BÀI TẬP TÌM KIẾM TRONG AI

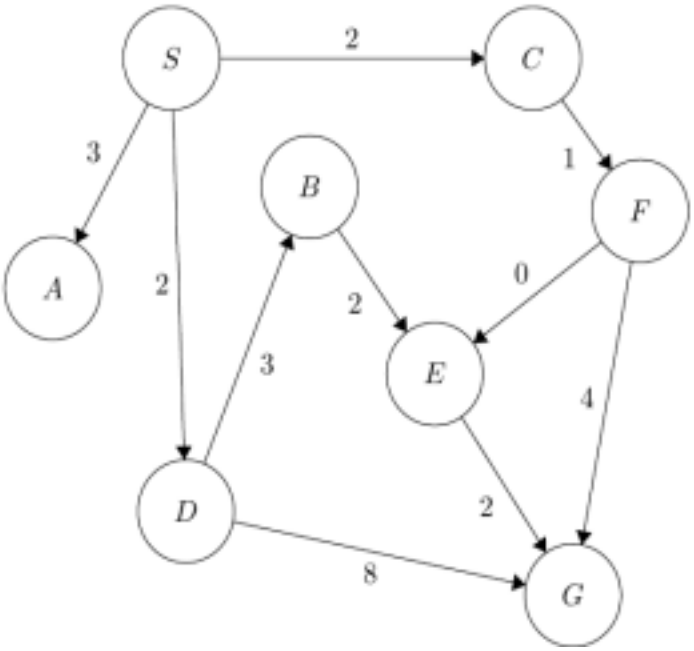
Họ và tên	Trần Ngọc Ánh
MSSV / Lớp	3122410013_ DCT1221

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
• Thiết kế và phân tích heuristic cho A*.
• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	{S, A, C, D, F, B}
Path	S → D → G

Total Cost	10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	{S, A, C, F, E}
Path	S → C → F → E → G
Total Cost	5

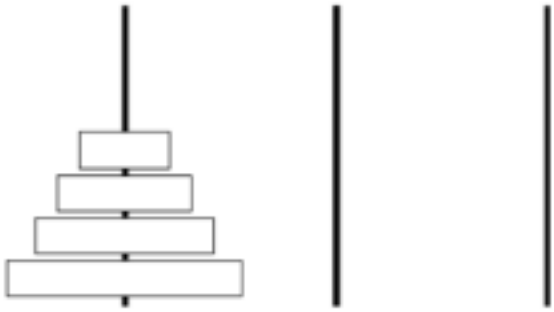
1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G
...	Đã tìm thấy G

Path cuối cùng: S → D → G
Total Cost: 10
Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?
- BFS: Sử dụng hàng đợi (Queue – FIFO), duyệt các nút theo chiều rộng. BFS đảm bảo tìm được đường đi ít cạnh nhất, nhưng không đảm bảo tổng chi phí nhỏ nhất khi các cạnh có trọng số khác nhau.
- DFS: Sử dụng ngăn xếp (Stack – LIFO), ưu tiên đi sâu theo một nhánh. DFS thường tốn ít bộ nhớ hơn BFS nhưng không đảm bảo tìm được đường đi tối ưu.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

- Ta biểu diễn một trạng thái bằng bộ ba (A, B, C) trong đó A, B, C lần lượt là danh sách các đĩa trên ba cọc. Các đĩa trong mỗi danh sách được sắp xếp từ dưới lên trên.
- Ví dụ với 3 đĩa, trạng thái ban đầu được biểu diễn là: ([3, 2, 1], [ ], [ ])
- Trong đó đĩa 3 là đĩa lớn nhất và nằm dưới cùng, đĩa 1 là đĩa nhỏ nhất và nằm trên cùng. Cách biểu diễn này cho biết vị trí của từng đĩa và xác định được đĩa nào đang nằm trên cùng của mỗi cọc để kiểm tra các hành động hợp lệ.

2.2. Kích thước không gian trạng thái

- Với n đĩa, mỗi đĩa có thể nằm trên một trong ba cọc A, B hoặc C. Vì vậy, mỗi đĩa có 3 lựa chọn vị trí.
- Theo quy tắc nhân, số trạng thái có thể có là: $|S| = 3 \times 3 \times ... \times 3 = 3^n$.
- Do đó, không gian trạng thái có kích thước là 3^n.
- Thứ tự các đĩa trên mỗi cọc được xác định bởi kích thước của chúng: đĩa lớn luôn nằm dưới đĩa nhỏ.

2.3. Trạng thái bắt đầu

- Ban đầu, tất cả n đĩa được xếp trên cọc A, đĩa lớn nằm dưới và đĩa nhỏ nằm trên. Theo cách biểu diễn đã chọn, trạng thái bắt đầu là:

S_0 = ([n, n-1, ..., 2, 1], [], []).

Ví dụ với 3 đĩa:

S_0 = ([3, 2, 1], [], []).

2.4. Các hành động hợp lệ

- Từ một trạng thái s, một hành động hợp lệ là di chuyển đĩa nằm trên cùng của một cọc sang một trong hai cọc còn lại.
- Một hành động phải thỏa mãn hai điều kiện:
    1. Chỉ được di chuyển đĩa nằm trên cùng của cọc nguồn.
    2. Không được đặt đĩa lớn lên trên đĩa nhỏ. Nếu cọc đích rỗng thì có thể đặt đĩa vào. Nếu cọc đích không rỗng thì đĩa được di chuyển phải nhỏ hơn đĩa đang nằm trên cùng của cọc đích.
- Các hướng di chuyển có thể là: A → B, A → C, B → A, B → C, C → A, C → B với điều kiện hành động thỏa mãn hai quy tắc trên.

2.5. Goal Test

- Hàm goalTest(state) trả về True khi tất cả n đĩa đã được chuyển sang cọc C.
- Theo cách biểu diễn đã chọn, trạng thái đích là:

G = ([], [], [n, n-1, ..., 2, 1]).

Ví dụ với 3 đĩa:

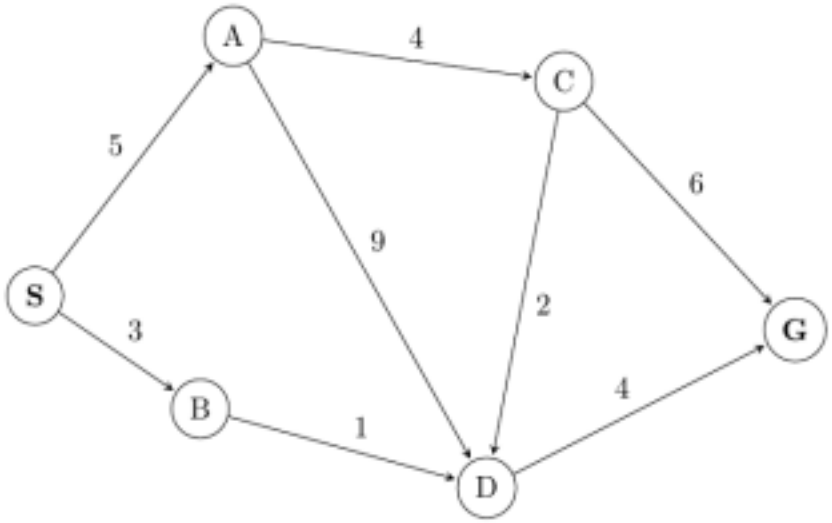
G = ([], [], [3, 2, 1]).

- Do đó: goalTest(state) = True nếu state = G; ngược lại trả về False.

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây.

Hãy thiết kế heuristic và phân tích ảnh hưởng heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Ta chọn $h(n)$ là chi phí đường đi ngắn nhất từ mỗi nút đến nút đích G.

- Với G: $h(G) = 0$
- Với D: $D \to G = 4$ nên $h(D) = 4$
- Với C: $C \to D \to G = 2 + 4 = 6$ nên $h(C) = 6$
- Với B: $B \to D \to G = 1 + 4 = 5$ nên $h(B) = 5$
- Với A: $A \to C \to D \to G = 4 + 2 + 4 = 10$ nên $h(A) = 10$
- Với S: $S \to B \to D \to G = 3 + 1 + 4 = 8$ nên $h(S) = 8$

Node	h(n)
S	8
A	10

B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Công thức: $f(n) = g(n) + h(n)$

Trong đó:

$g(n)$: chi phí từ nút bắt đầu S đến nút n

$h(n)$: chi phí ước lượng từ n đến đích G

$f(n)$: tổng chi phí dùng để lựa chọn nút tiếp theo.

Bước 1: Xét nút S

Tại S:

$g(S) = 0,\ h(S) = 8$

$f(S) = 0 + 8 = 8$

Từ S có thể đi đến A và B.

Với A:

$g(A) = 5$

$f(A) = 5 + 10 = 15$

Với B:

$g(B) = 3$

$f(B) = 3 + 5 = 8$

Chọn B vì: $f(B) = 8 < f(A) = 15$

Bước 2: Xét nút B

Từ B đi đến D với chi phí 1:

$g(D) = g(B) + 1 = 3 + 1 = 4$

$f(D) = 4 + 4 = 8$

So sánh các nút đang xét:

A: $f(A) = 15$

D: $f(D) = 8 \rightarrow$ Chọn D.

Bước 3: Xét nút D

Từ D có thể đi đến C và G.

Với C:

$g(C) = 4 + 2 = 6$

$f(C) = 6 + 6 = 12$

Với G:

$g(G) = 4 + 4 = 8$

$f(G) = 8 + 0 = 8$

Chọn G vì $f(G) = 8$, đồng thời G là nút đích nên kết thúc tìm kiếm.

Kết quả

Explored Set: $S \to B \to D \to G$

Path: $S \to B \to D \to G$

Total Cost: 3 + 4 + 1 = 8

Vậy đường đi tìm được bằng thuật toán A* là: $S \to B \to D \to G$ với tổng chi phí bằng 8.

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Chọn một bộ giá trị heuristic mới:

Node	h(n)
S	8
A	10
B	0
C	6
D	4
G	0

Các giá trị này vẫn admissible vì:

$h(S) = 8 \leq 8$

$h(A) = 10 \leq 10$

$h(B) = 0 \leq 5$

$h(C) = 6 \leq 6$

$h(D) = 4 \leq 4$

$h(G) = 0 \leq 0$

Tuy nhiên, heuristic này không consistent.

Xét cạnh $S \to B$, có: $c(S, B) = 3$

Điều kiện consistent yêu cầu: $h(S) \leq c(S, B) + h(B)$

Thay các giá trị vào: $8 \leq 3 + 0$

$8 \leq 3 \to$ Điều này sai.

Vì vậy cạnh $S \to B$ vi phạm điều kiện consistent: h(S) > c(S,B) + h(B)

Do đó, bộ heuristic trên là admissible nhưng không consistent.

3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Heuristic consistent thỏa mãn điều kiện: $h(n) \leq c(n, n') + h(n')$ với mọi cạnh từ n đến n'.

Xét một đường đi từ n đến đích G: $n \to n_1 \to n_2 \to \ldots \to G$

Áp dụng điều kiện consistent cho từng cạnh: $h(n) \le c(n, n_1) + h(n_1)$

$$h(n_1) \le c(n_1, n_2) + h(n_2)$$

Tiếp tục cho đến nút G, ta có: $h(n) \le c(n, n_1) + c(n_1, n_2) + \ldots + c(n_k, G) + h(G)$

Do giả sử: $h(G) = 0$

nên: $h(n) \le c(n, n_1) + c(n_1, n_2) + \ldots + c(n_k, G)$

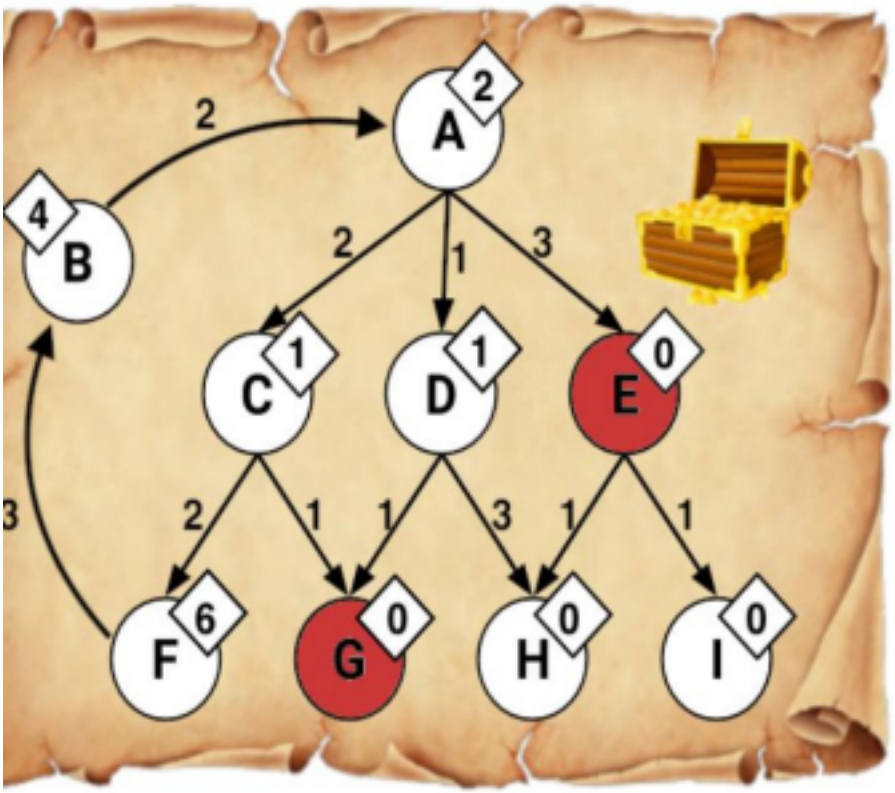
Vế phải chính là chi phí của một đường đi từ n đến G. Vì đường đi ngắn nhất có chi phí nhỏ nhất nên: $h(n) \le h^*(n)$

Trong đó $h^*(n)$ là chi phí thực tế nhỏ nhất từ n đến G.

Vì vậy, $h(n)$ không bao giờ đánh giá lớn hơn chi phí thực tế tối ưu. Do đó heuristic consistent cũng thỏa mãn tính admissible. Consistent → Admissible

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: Khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search
Explored Set: A, C, F, B, G
Path returned: A → C → G

4.2. Breadth-First Search
Explored Set: A, C, D, E
Path returned: A → E

4.3. Uniform-Cost Search
Explored Set: A, D, C, G
Path returned: A → D → G
Cost: 2

4.4. Greedy Search
Explored Set: A, E
Path returned: A → E
Cost: 3

4.5. A* Search
Explored Set: A, D, G
Path returned: A → D → G
Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A → C → G	3
BFS	Độ sâu nhỏ nhất	A → E	3
UCS	g(n)	A → D → G	2
Greedy	h(n)	A → E	3
A*	g(n)+h(n)	A → D → G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: Sai

Giải thích: Thuật toán tìm kiếm theo chiều sâu (DFS) hoàn toàn có thể mở rộng ít nút hơn so với A*. Lý do là DFS có thể may mắm đi thẳng một mạch xuống nhánh chứa trạng thái đích ngay từ những bước đầu tiên (dù đường đi đó có thể không tối ưu) và thuật toán sẽ dừng lại ngay khi tìm thấy đích. Trong khi đó, thuật toán A* bắt buộc phải đảm bảo tính tối ưu của đường đi.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai:Sai

Giải thích: Hàm heuristic $h(n)$ chỉ admissible khi không đánh giá cao hơn chi phí thực tế ($h(n) \leq h^*(n)$). Trong cờ vua, chi phí tính theo số nước đi và quân xe có thể lướt qua nhiều ô trong đúng 1 nước đi. Ví dụ: đi từ ô $(1, 1)$ đến $(1, 8)$ trên bàn cờ trống chỉ tốn chi phí thực tế $h^* = 1$ nước đi, nhưng khoảng cách Manhattan lại ước lượng là |1 - 1| + |8 - 1| = 7. Vì $h = 7 > h^* = 1$ (overestimate), nên khoảng cách Manhattan không phải là admissible heuristic.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman. Đúng / Sai: Đúng

Giải thích: Một heuristic $h(n)$ là admissible nếu nó không bao giờ đánh giá cao hơn chi phí thực tế ($h(n) \leq h^*(n)$). Khoảng cách Euclid là đường chim bay (khoảng cách ngắn nhất giữa hai điểm trong không gian phẳng). Trong khi đó, Pacman di chuyển trên lưới ô vuông rời rạc (chỉ đi ngang/dọc) và có thể gặp tường chắn, nên độ dài đường đi thực tế $h^*(n)$ luôn lớn hơn hoặc bằng khoảng cách Euclid. Do đó, Euclidean distance là một admissible heuristic.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Hàm heuristic $h(n)$ chỉ admissible khi không đánh giá cao hơn chi phí thực tế ($h(n) \leq h^*(n)$). Giả sử hai hàm $h_1(n)$ và $h_2(n)$ đều admissible, khi cộng lại ta có $h(n) = h_1(n) + h_2(n)$ hoàn toàn có thể vượt quá $h^*(n)$. Ví dụ: chi phí thực tế $h^* = 10$, nếu chọn hai admissible heuristics hợp lệ là $h_1 = 6 \leq 10$ và $h_2 = 7 \leq 10$, thì tổng $h = 6 + 7 = 13 > 10$ (bị overestimate). Vì vậy, tổng của nhiều admissible heuristics không đảm bảo luôn là admissible.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích: Consistency luôn kéo theo admissibility, nhưng chiều ngược lại không đúng: một heuristic admissible chưa chắc đã consistent. Heuristic consistent bắt buộc phải thỏa bất đẳng thức tam giác $h(n) \leq c(n, a, n') + h(n')$.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: A^* dùng Tree Search chỉ cần heuristic admissible là tối ưu, nhưng với Graph Search thì bắt

buộc heuristic phải consistent (nhất quán).

Nếu heuristic chỉ admissible mà không consistent, tập các nút đã duyệt (closed list) của Graph Search sẽ chặn không cho cập nhật lại (re-open) các nút này khi tìm thấy đường đi mới tốt hơn, dẫn đến nguy cơ bỏ sót đường đi tối ưu.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

---o0o---

(Hết)